PROJECT DOCUMENTATION

ON

SCRIPT-CONTROLLED ACL

Restrict Record Access Based on Custom Script

Submitted in partial fulfillment of the requirements for the Naan Mudhalvan Skill Development Programme

NAAN MUDHALVAN
Skill Development Programme

Submitted By
Team ID: SWTID-2026-5170
Date: 02 October 2026

S.No.	Team member name	Role
1	Shanmugapriya B	Team Lead - Project planning, architecture and ACL scripting
2	Shrivinoj Karthikeyan	Member - Tables creation and READ / CREATE ACLs
3	Mohammed Ameen S	Member - WRITE / DELETE ACLs and testing
4	Srinithi T	Member - Users and roles setup, documentation

Submitted To
[Enter College Name]

INDEX

Table of Contents

1. Introduction 4

Project title 4

Team members 4

2. Project Overview 5

2.1 Purpose 5

2.2 Features 5

3. Architecture 6

3.1 System architecture 6

3.2 Platform 6

3.3 ACL engine 6

3.4 Database 6

4. Setup Instructions 7

4.1 Prerequisites 7

4.2 Installation and configuration 7

5. Project Structure 8

5.1 Logical project structure 8

6. Running the Application 9

6.1 Steps to run the project 9

6.2 Expected output 9

6.3 ACL scripts used 9

7. API Documentation 10

7.1 API integration 10

7.2 Core objects used in the scripts 10

7.3 Checking access from a script 10

7.4 Optional REST API 10

8. Authentication and Authorization 11

8.1 Authentication 11

8.2 Authorization 11

8.3 Security 11

9. User Interface 12

9.1 Main interface components 12

Screenshots 12

10. Testing 14

10.1 Testing types 14

10.2 Test results 14
10.3 Demonstration steps 14
10.4 Demo video 14
10.5 GitHub repository 14
11. Known Issues 15
12. Future Enhancements 15
Conclusion 16

1. Introduction

Project title

Script-Controlled ACL – Restrict Record Access Based on Custom Script

Team members

S.No.	Team member name	Role
1	Shanmugapriya B	Team Lead - Project planning, architecture and ACL scripting
2	Shrivinoj Karthikeyan	Member - Tables creation and READ / CREATE ACLs
3	Mohammed Ameen S	Member - WRITE / DELETE ACLs and testing
4	Srinithi T	Member - Users and roles setup, documentation

2. Project Overview

2.1 Purpose

The purpose of this project is to restrict access to records in ServiceNow using **script-controlled Access Control Lists (ACLs)**. A script checks specific field values and user conditions before granting access, and users can access a record only when the ACL script evaluates to **true**.

Standard role-based ACLs give a role access to every record in a table. This project adds record-level control so that confidential data stays protected and users can work only with the records they are allowed to.

2.2 Features

- **Script-based decisions:** a custom script decides if access is allowed.
- **Record-level security:** access depends on field values such as confidential, owner and status.
- **User-based conditions:** roles and the logged-in user are checked.
- **Full operation coverage:** separate ACLs for Read, Create, Write and Delete.
- **Server-side enforcement:** rules apply to lists, forms and REST access.
- **Testable:** behaviour is verified by impersonating users.

3. Architecture

3.1 System architecture

The project is built entirely inside the ServiceNow platform; there is no separate frontend or backend. The main components are:

- **User:** signs in and requests an operation on a record.
- **ServiceNow platform:** authenticates the user and loads roles.
- **ACL engine:** finds the ACLs for the table and operation and evaluates them.
- **ACL script:** returns true or false using roles, user ID and record fields.
- **Custom table:** stores the protected records.

3.2 Platform

ServiceNow (Personal Developer Instance) provides user management, tables, ACLs and the impersonation feature used for testing.

3.3 ACL engine

An ACL grants access only when its **role requirement**, **condition** and **script** are all satisfied. The script sets a variable named `answer` to true to allow access or false to deny it.

3.4 Database

Table / component	Purpose
u_confidential_record	Custom table that holds the protected records (name, department, status, confidential, owner).
sys_user	Stores the project users.
sys_user_role / sys_user_has_role	Store the roles and their assignment to users.
sys_security_acl	Stores the ACL definitions and scripts.

The project uses built-in ServiceNow tables and does not need an external database.

4. Setup Instructions

4.1 Prerequisites

- A ServiceNow Developer account and a Personal Developer Instance.
- A modern web browser such as Google Chrome or Microsoft Edge.
- Admin access to the instance and permission to elevate to the security_admin role.
- Basic knowledge of tables, roles and ACLs; internet connectivity.

4.2 Installation and configuration

Step 1 - Access the developer instance. Sign in at the ServiceNow Developer Portal and open your instance.

Step 2 - Milestone 1: Creation of users and roles.

1. Open User Administration > Users and create the project users (for example record.admin, record.owner, record.user and record.norole).
2. Open User Administration > Roles and create the roles `u_record_admin` and `u_record_user`.
3. Assign u_record_admin to the admin user, and u_record_user to the owner and standard user. Leave the no-role user without project roles.

Step 3 - Milestone 2: Tables creation.

1. Open System Definition > Tables > New. Label: Confidential Record; Name: u_confidential_record.
2. Add the columns: Name (String), Department (String), Status (Choice: Draft, Active, Closed), Confidential (True/False), Owner (Reference to User).
3. Clear the option that auto-creates default access controls so that only your own ACLs apply, then submit.
4. Create sample records: some confidential, some not, owned by different users.

Step 4 - Milestones 3 to 6: Creation of ACLs (READ, CREATE, WRITE, DELETE).

1. Elevate your role to security_admin (user menu > Elevate role).
2. Open System Security > Access Control (ACL) > New.
3. Set Type = record, the Operation (read, create, write or delete) and select the Confidential Record table (leave the field as None for a table-level rule).
4. Select **Advanced** and paste the script for that operation (see Section 6 and the scripts below), then submit.
5. Repeat for the other three operations.

Step 5 - Milestone 7: Test and conclude. Use Impersonate User for each account, run the checks in Section 10 and record the results.

5. Project Structure

Unlike a traditional application, this project has no client and server folders. The work is configured inside ServiceNow and the GitHub repository holds the documentation in the following structure.

5.1 Logical project structure

```
Script-Controlled ACL - ServiceNow/
|
|-- 1. Ideation Phase/
|   |-- Brainstorm___Idea_Prioritization.pdf
|   |-- Define_The_Problem_Statement.pdf
|   +-- Empathy_Map_Canvas.pdf
|
|-- 2. Requirement Analysis/
|   |-- Customer Journey Map.pdf
|   |-- Data Flow Diagrams and User Stories.pdf
|   |-- Solution Requirements.pdf
|   +-- Technology Stack.pdf
|
|-- 3. Project Design Phase/
|   |-- Problem Solution Fit/Problem - Solution Fit.pdf
|   |-- Proposed Solution/Proposed Solution.pdf
|   +-- Solution Architecture/Solution Architecture.pdf
|
|-- 4. Project Planning Phase/
|   |-- Project Logic.pdf
|   +-- Project Planning.pdf
|
|-- 5. Project Development Phase/
|   |-- Performance Testing/
|   |   |-- Functional and Performance Testing.pdf
|   |   |-- ACL Script Performance Test.pdf
|   |   +-- User Acceptance Testing.pdf
|   +-- User Acceptance Testing/UAT Report.pdf
|
|-- 6. Project Documentation/
|   |-- Project Documentation.pdf
|   +-- Project Report.pdf
|
+-- README.md
```

The actual tables, roles and ACLs are maintained in your ServiceNow instance.

6. Running the Application

There are no local servers to start because the project runs on the ServiceNow platform.

6.1 Steps to run the project

1. Log in to your ServiceNow developer instance as an administrator.
2. Confirm that the four ACLs (read, create, write, delete) are active on the Confidential Record table.
3. Open the user menu and choose Impersonate User, then select a project user.
4. Open the Confidential Record list and try to view, create, edit and delete records.
5. End impersonation and repeat for the next user.

6.2 Expected output

Each user can perform only the operations allowed by the access matrix below.

User / Role	Read	Create	Write	Delete
Admin (u_record_admin)	All records	Yes	All records	Yes
Owner (u_record_user, owns the record)	Own records (incl. confidential)	Yes	Own record, until Closed	No
Other user (u_record_user, not owner)	Non-confidential only	Yes	No	No
User without project roles	No	No	No	No

6.3 ACL scripts used

READ ACL script

```
answer = false;
if (gs.hasRole('u_record_admin')) {
    answer = true;                      // admins can read every record
} else if (current.u_owner == gs.getUserID()) {
    answer = true;                      // owners can read their own records
} else if (gs.hasRole('u_record_user') && current.u_confidential == false) {
    answer = true;                      // users can read non-confidential records
}
```

CREATE ACL script

```
answer = gs.hasRole('u_record_admin') || gs.hasRole('u_record_user');
```

WRITE ACL script

```
answer = false;
if (gs.hasRole('u_record_admin')) {
    answer = true;
} else if (current.u_owner == gs.getUserID() && current.u_status != 'closed') {
    answer = true;                      // owner may edit until the record is closed
}
```

DELETE ACL script

```
answer = gs.hasRole('u_record_admin');
```

7. API Documentation

7.1 API integration

The project uses the ServiceNow server-side Glide API inside ACL scripts. No separate API was developed.

7.2 Core objects used in the scripts

Object / method	Description
answer	Variable set to true to allow access or false to deny it.
current	The record being accessed; fields are read as current.u_owner and so on.
gs.hasRole('role')	Returns true if the logged-in user has the given role.
gs.getUserID()	Returns the sys_id of the logged-in user.

7.3 Checking access from a script

```
var gr = new GlideRecord('u_confidential_record');
gr.query();
while (gr.next()) {
    gs.info(gr.u_name + ' read=' + gr.canRead() + ' write=' + gr.canWrite() + ' delete=' + gr.canDelete());
}
```

7.4 Optional REST API

The ServiceNow Table API (`/api/now/table/u_confidential_record`) also enforces the ACLs, so a REST client sees only the records it is permitted to read. External API integration is not required for this project.

8. Authentication and Authorization

8.1 Authentication

- Users sign in to the ServiceNow instance with authorised credentials.
- Authentication is handled by the platform.

8.2 Authorization

Role / user	Access
u_record_admin	Full control: read, create, write and delete.
u_record_user (owner)	Read, edit (until Closed) and create their own records.
u_record_user (other)	Read non-confidential records and create records.
No project role	No access to the table.

8.3 Security

Access control is enforced on the server by ACLs. Scripts default to *answer* = *false*, so any case that is not explicitly allowed is denied.

9. User Interface

The project uses the standard ServiceNow interface.

9.1 Main interface components

1. User Administration - users and roles.
2. Tables - the Confidential Record table definition.
3. Access Control (ACL) - the four ACL records.
4. Confidential Record list and form - what each user sees.
5. Impersonate User - used to verify each role.

Screenshots

Insert screenshot: Users and roles created (Milestone 1)

Figure 1: Users and roles

Insert screenshot: Confidential Record table and columns (Milestone 2)

Figure 2: Table creation

Insert screenshot: Four ACL records for the table (Milestones 3-6)

Figure 3: Access control list

Insert screenshot: List view as admin versus standard user

Figure 4: Read access result

Insert screenshot: Write/Delete denied for a standard user

Figure 5: Write and delete restriction

10. Testing

Testing verified that every operation behaves as defined in the access matrix.

10.1 Testing types

- **Functional testing:** Read, Create, Write and Delete for each user type.
- **Performance testing:** list loading with 50+ and 500+ records; repeated access.
- **User acceptance testing (UAT):** confirm the rules meet the stated requirements.

10.2 Test results

Testing type	Total test cases	Passed	Failed
Functional testing	10	[ ]	[ ]
Performance testing	3	[ ]	[ ]
Total	**13**	**[ ]**	**[ ]**

Fill in the Passed and Failed columns from the detailed test sheets in the Project Development Phase folder.

10.3 Demonstration steps

1. Open the ServiceNow developer instance.
2. Show the users, roles and table.
3. Show the four ACLs and their scripts.
4. Impersonate each user and show the list and form behaviour.
5. Show Create, Write and Delete being allowed or denied.

10.4 Demo video

[Paste demo video link here]

10.5 GitHub repository

[Paste GitHub repository link here]

11. Known Issues

The following points apply to the current implementation and are considerations rather than confirmed defects:

- Users with the *admin* role pass role checks for every role, so use non-admin test accounts and impersonation.
- Role names and field names in the scripts must match those in your instance exactly.
- A script that throws an error results in denied access, so scripts should be kept simple.
- Changing the table design (for example renaming a field) requires updating the scripts.
- A developer instance may hibernate when it is not used for a long time.

12. Future Enhancements

1. **Field-level ACLs** to hide or lock individual fields such as salary or notes.
2. **Reusable Script Include** that holds the access logic so all ACLs share one function.
3. **Group-based rules** that grant access to members of a department or group.
4. **Audit logging** of denied attempts for security review.
5. **Approval workflow** before a Closed record can be reopened.
6. **Reporting** on who accessed which records.

Conclusion

The Script-Controlled ACL project shows how ServiceNow ACLs can be extended with custom scripts to protect records beyond what roles alone allow. By checking field values and user conditions, the scripts decide whether a user may read, create, write or delete a record and grant access only when the result is true.

The project was organised in seven milestones, from users and roles to the final conclusion, and verified by impersonating users against an access matrix. It provides a base that can be extended with field-level ACLs, shared Script Includes and audit logging.